

REVISÃO PARA A A1

Estrutura de Dados — Aulas 01 a 07

Prof. Alysson

Prova: sexta-feira, 09/10/2026

Como usar esta revisão

- São **9 etapas**, uma por assunto, mais o **simulado**
- O texto de revisão traz, em cada etapa:
 - resumo direto, **exemplo resolvido**, armadilhas
 - **perguntas de fixação** com "Confira a resposta"
- Estes slides são o **resumo visual**: use-os para lembrar e para revisar de novo na véspera
- Tempo total: **cerca de 3 horas** — divida em 2 ou 3 dias
- Estude com **papel e caneta** ao lado

O caminho

Etapa	Assunto	Tempo
0	Mapa da prova e autoavaliação	5 min
1	<code>javac</code> , <code>java</code> , classpath e pacotes	10 min
2	Classe abstrata, interface, <code>this</code> , polimorfismo	15 min
3	Complexidade: contar as voltas	20 min
4	Recursão e pilha de chamadas	15 min
5	Arquivos e TAD	15 min
6	Vetor dinâmico × lista encadeada	20 min
7	Pilha, fila e fila circular	15 min
8	Ordenação simples	20 min
9	Simulado sem consulta	45 min

Etapa 0

Mapa da prova

A prova A1

- **Data:** sexta-feira, **09/10/2026**
- Individual, escrita, **em papel** e **sem consulta** — vale **10,0**
- **Parte I:** 10 objetivas (A, B, C, D) $\times 0,5 = \mathbf{5,0}$
- **Parte II:** 1 discursiva = **5,0**, escrita à mão
- **Cai:** Aulas 01 a 07
- **Não cai:** Merge Sort, Quick Sort (Aula 08), árvores, grafos

Os quatro formatos de objetiva

Formato	Como atacar
Traçado de código — "o que é impresso?"	execute no papel, linha por linha
Afirmações I a IV	V ou F em cada uma, sozinha ; depois a alternativa
Asserção-razão (PORQUE)	julgue I, julgue II; só então: II explica I?
Escolha + justificativa	decisão e motivo têm de estar certos

Muitas questões pedem para acompanhar código à mão: treine com lápis!

A discursiva

- Escrever **à mão** um método sobre **lista, pilha ou fila**
- Dar a **complexidade** em notação O, **com justificativa**
- Critérios de correção:
 - i. identificar o problema e os **casos especiais** (estrutura vazia!)
 - ii. **assinatura** certa, usando os campos dados
 - iii. **lógica** principal correta
 - iv. **complexidade** justificada pelo número de passos
 - v. **clareza**

Autoavaliação: "eu sei..."

Eu sei...	Se não, volte à
escrever o <code>java -cp ...</code> de uma classe em pacote	Etapa 1
por que <code>new</code> de classe abstrata não compila; o que faz <code>this.nome = nome</code>	Etapa 2
contar voltas com <code>i += 3</code> , laço interno até <code>i</code> , laços em sequência	Etapa 3
estimar o tempo quando n é multiplicado por 3 ou 4	Etapa 3
o que um método recursivo imprime antes e depois da chamada	Etapa 4
pré, pós e invariante: quem garante cada um	Etapa 5
o custo do k-ésimo em <code>ArrayList</code> e em lista encadeada	Etapa 6
o efeito de uma linha trocada num método de lista	Etapa 6
<code>inicio</code> , <code>fim</code> e <code>tam</code> na fila circular, com a volta para 0	Etapa 7
o vetor após uma passagem de Bubble, Selection e Insertion	Etapa 8

Etapa 1

javac, **java**, classpath e pacotes

Quem faz o quê

- **javac** = compilador: `.java` → `.class`
- **java** = executa uma **classe** (nunca escreva `.class` nem `.java`)
- **package br.unitins.revisao;** → nome completo
`br.unitins.revisao.Ola`
- **-cp** = a **raiz** onde a JVM começa a procurar
- **javac -d pasta** recria as **pastas do pacote** dentro de `pasta`

Exemplo resolvido

```
javac -d classes fontes/br/unitins/revisao/Ola.java  
java -cp classes br.unitins.revisao.Ola
```

- o `.class` vai para `classes/br/unitins/revisao/Ola.class`
- a JVM junta `classes` + `br/unitins/revisao/` + `Ola.class`
- **Regra:** `-cp` aponta para a **raiz**; o nome da classe leva o **pacote**

Armadilhas (todas testadas — todas falham)

Comando	Erro
<code>java -cp classes Ola</code>	falta o pacote no nome
<code>java -cp classes br.unitins.revisao.Ola.class</code>	<code>java</code> não leva <code>.class</code>
<code>java -cp classes/br/unitins/revisao Ola</code>	<code>-cp</code> deve ser a raiz, não a pasta do pacote
<code>javac -cp classes br.unitins.revisao.Ola</code>	<code>javac</code> compila arquivo; quem executa é <code>java</code>

Sem `-d`, o `.class` fica ao lado do `.java`, na pasta dos fontes.

Etapa 2

**Classe abstrata, interface, `this` e
polimorfismo**

O essencial

- **Interface** = contrato: o *que* faz. Quem faz `implements` escreve **todos** os métodos
- **Classe abstrata** = incompleta: **não** existe `new ClasseAbstrata()`
 - mas **pode** ser tipo de variável e de vetor
- Se a abstrata não escreveu o método do contrato, a **primeira subclasse concreta** é obrigada
- `extends` só **uma** classe; `implements` **várias** interfaces
- `this.nome = nome` guarda o parâmetro no atributo
- `nome = nome` **compila**, mas o atributo fica `null`

Polimorfismo: quem decide o quê

- **Tipo declarado** da variável → **quais** métodos posso chamar
- **Tipo real** do objeto (o do `new`) → **qual versão** executa

```
Instrumento[] palco = { new Violao("Lia"), new Tambor("Davi") };  
for (Instrumento x : palco) {  
    System.out.println(x.getNome() + ": " + x.som());  
}
```

Saída: `Lia: dlim` e `null: tum` — o `Tambor` usou `nome = nome` sem `this`

Armadilhas da Etapa 2

Código	Compila?
<code>new Instrumento()</code> (abstrata)	não — <code>cannot be instantiated</code>
<code>Sonoro s = new Violao("Téo"); s.getNome();</code>	não — <code>Sonoro</code> só conhece <code>som()</code>
subclasse concreta sem <code>som()</code>	não — <code>does not override abstract method</code>
<code>nome = nome;</code> no construtor	sim — e o atributo fica <code>null</code>
método sem <code>@Override</code>	sim — <code>@Override</code> só pede conferência

Passagem de parâmetro: Java passa **cópia da referência**. O método muda o **estado** do objeto, mas não troca a variável de quem chamou.

Etapa 3

Complexidade: contar as voltas

Três passos

1. Ache o laço que roda **mais vezes**
2. **Conte** as voltas em função de n
3. **Jogue fora** constantes e termos menores

Contagem	O	$n \times k \rightarrow$ tempo
7, 100 (fixo)	$O(1)$	igual
$\sim \log_2 n$	$O(\log n)$	+ poucas voltas
$n/3, 2n, n + 10$	$O(n)$	$\times k$
$n^2/2, n(n - 1)/2$	$O(n^2)$	$\times k^2$

As cinco armadilhas

Trecho	Voltas	O
for (i = 0; i < n; i += 3)	$\sim n/3$	$O(n)$
interno for (j = 0; j < i; j++)	$n(n - 1)/2$	$O(n^2)$
interno for (j = n / 2; j < n; j++)	$n^2/2$	$O(n^2)$
interno for (j = 0; j < 3; j++)	$3n$	$O(n)$
laço até n depois de outro laço até n	$2n$	$O(n)$
for (i = 1; i < n; i *= 2)	$\sim \log_2 n$	$O(\log n)$

Aninhado multiplica; **em sequência** soma. Metade de n^2 ainda é n^2 .

Estimando o tempo

Trecho $O(n^2)$ levou 0,5 s com $n = 4.000$. E com $n = 12.000$?

1. Fator de n : $12.000 \div 4.000 = 3$

2. $O(n^2) \rightarrow \text{trabalho} \times 3^2 = 9$

3. Tempo: $0,5 \times 9 = 4,5 \text{ s}$

Se fosse $O(n)$: $0,5 \times 3 = 1,5 \text{ s}$. Se fosse $O(1)$: 0,5 s.

Cuidado: "metade de n " aparece nos dois tamanhos e **se cancela** — não divide o fator.

Etapa 4

Recursão e pilha de chamadas

O essencial

- **Caso base:** responde direto, **sem** nova chamada
- **Passo recursivo:** chama com problema **menor**, rumo ao caso base
- Sem caso base → `StackOverflowError`
- Cada chamada = um **quadro** na pilha de chamadas
- **Regra do traçado:**
 - o que vem **antes** da chamada roda na **ida**
 - o que vem **depois** roda na **volta**, de trás para a frente

Exemplo resolvido

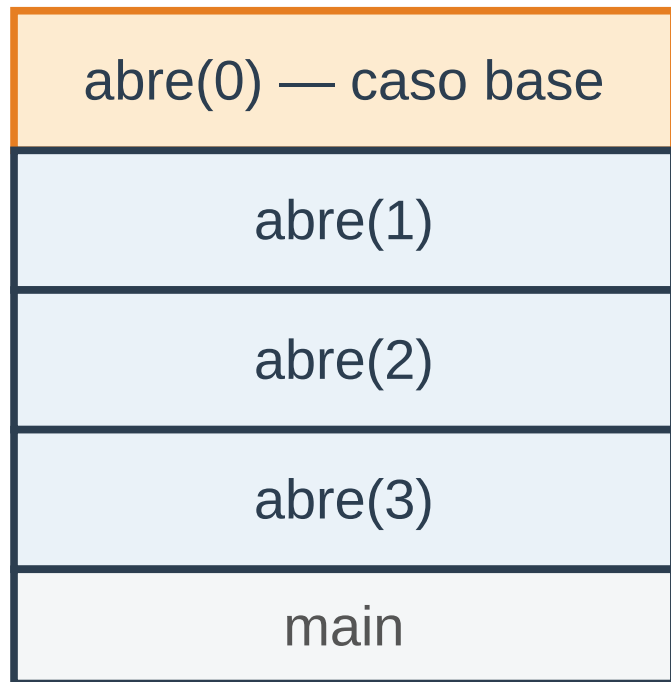
```
static void abre(int n) {  
    if (n == 0) { System.out.print("* "); return; } // caso base  
    System.out.print("(" + n + " "); // ida  
    abre(n - 1); // passo recursivo  
    System.out.print(n + ") "); // volta  
}
```

abre(3) imprime: (3 (2 (1 * 1) 2) 3)

Cada chamada **abre** na ida e **fecha** na volta.

A pilha de `abre(3)`

Pilha de chamadas de `abre(3)`



IDA (empilha, de baixo para cima):

imprime (3 (2 (1 e o caso base imprime *

VOLTA (desempilha, de cima para baixo):

imprime 1) 2) 3)

Saída: (3 (2 (1 * 1) 2) 3)

O que vem DEPOIS da chamada só roda na volta.

Armadilhas da Etapa 4

- Impressão **depois** da chamada sai **invertida**: quem imprime só na volta mostra `1 2 3 4` para $n = 4$
- O **caso base** também pode imprimir — não esqueça dele no meio da saída
- Passo que **pula** o caso base: `n - 3` com caso base `n == 0` nunca para em $7 \rightarrow 4 \rightarrow 1 \rightarrow -2...$
- **Valor devolvido** é montado na **volta**: `dobra(4)` = $2 \times 2 \times 2 \times 2 \times 1 = 16$
- Recursão **não** é mais rápida que laço: fatorial é $O(n)$ nos dois, mas a recursiva gasta $O(n)$ de pilha

Etapa 5

Arquivos em Java e TAD

Arquivos: o essencial

- Ler: `BufferedReader` + `FileReader`; `readLine()` devolve `null` só no fim
- Linha em branco chega como `""`, não como `null`
- Cada `readLine()` consome uma linha — uma chamada por volta
- `try-with-resources` fecha o arquivo ao sair do bloco, mesmo com exceção
- `FileWriter(..., UTF_8)` apaga; com `true` no fim, acrescenta
- Informe `StandardCharsets.UTF_8` — senão "João"
- Linha a linha gasta memória de uma linha, não do arquivo inteiro

O laço de leitura certo

```
try (BufferedReader leitor = new BufferedReader(  
    new FileReader("inscritos.txt", StandardCharsets.UTF_8))) {  
    String linha;  
    while ((linha = leitor.readLine()) != null) {  
        total++;  
        if (linha.isBlank()) vazias++;  
    }  
}
```

Arquivo com 6 linhas, uma em branco: **linhas=6 vazias=1**

Parando na linha vazia, leria só as 2 primeiras.

TAD: o contrato

- **TAD** = **o que** faz (interface), não **como** (classe)
- Trocar a implementação **não muda** o código que usa o TAD

	Quem garante	Quando
Pré-condição	quem chama	antes
Pós-condição	quem implementa	depois
Invariante	quem implementa	sempre, entre operações

- Dentro do método, o invariante pode quebrar **por um instante**
- **JavaDoc** (`@param` , `@return` , `@throws` , `@pre` , `@pos`) **documenta**; o compilador **não confere** — quem confere é o `if` + `throw`

Etapa 6

Vetor dinâmico × lista encadeada

Custos

Operação	ArrayList	Lista simples (com cauda)	Lista dupla
k-ésimo elemento	O(1)	O(n)	O(n)
inserir no fim	O(1) amortizado	O(1)	O(1)
inserir no início	O(n)	O(1)	O(1)
remover o primeiro	O(n)	O(1)	O(1)
remover o último	O(1)	O(n) — achar o penúltimo	O(1)
buscar valor	O(n)	O(n)	O(n)

- **Array fixo:** tamanho que nunca muda + acesso por índice
- **Vetor dinâmico** cheio → **dobra** a capacidade (2 → 4 → 8) e copia

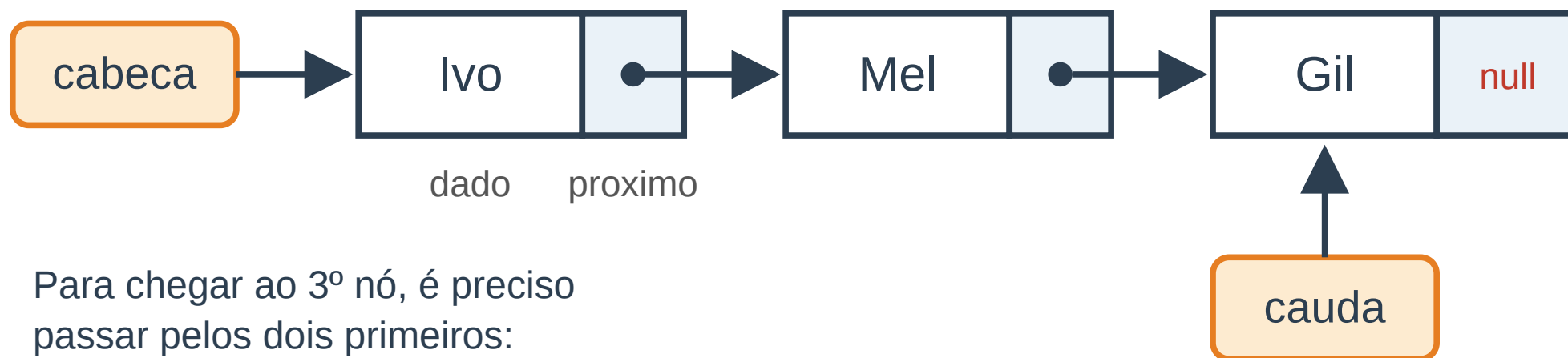
Traçado em lista

```
fila.inserirNoFim("Mel");  
fila.inserirNoFim("Gil");  
fila.inserirNoInicio("Ivo");    // Ivo → Mel → Gil → null [tamanho=3]  
fila.remover("Gil");           // Ivo → Mel → null [tamanho=2]  
fila.inserirNoFim("Zeca");     // Ivo → Mel → Zeca → null [tamanho=3]
```

- Removendo a **cauda**, o anterior (Mel) vira a nova cauda
- Sempre confira **cabeca**, **cauda** e **tamanho** a cada passo

A lista na memória

Lista encadeada simples: Ivo → Mel → Gil → null [tamanho=3]



Para chegar ao 3º nó, é preciso
passar pelos dois primeiros:
acesso por posição custa $O(n)$.

Defeito: a ordem das ligações importa

```
No<T> novo    = new No<>(dados);  
cabeca        = novo;           // a cabeça mudou primeiro...  
novo.proximo  = cabeca;         // ...e o nó aponta para si mesmo!
```

- Depois de inserir Mel e Gil: `Gil → Gil → Gil → ...` — o `imprimir()` **não para**
- O nó de Mel se **perdeu**, mas `tamanho` diz 2
- **Certo:** primeiro `novo.proximo = cabeca`, **depois** `cabeca = novo`

Como achar o efeito de um defeito

1. **Desenhe** nós e setas antes da primeira operação
2. Execute **uma linha por vez**, redesenhando cada seta
3. Confira **cabeca** , **cauda** e **tamanho** após cada chamada
4. Teste os **casos de borda**: lista **vazia**, **um** nó, operação na **cabeça** e na **cauda**
5. Veja o que o laço de percurso **examina**: o primeiro nó? o último?

Não confie no nome do método: execute o que está escrito.

Etapa 7

Pilha, fila e fila circular

Pilha (LIFO) e fila (FIFO)

- **Pilha:** último a entrar, **primeiro** a sair
 - `push`, `pop`, `peek`; `topo` começa em **-1**; `tamanho()` = `topo + 1`
 - usos: desfazer, parênteses, pilha de chamadas
- **Fila:** primeiro a entrar, primeiro a sair
 - `enqueue` no fim, `dequeue` na frente, `peek` só olha
 - usos: atendimento, impressão
- Todas as operações: **O(1)**
- Estrutura cheia ou vazia → `IllegalStateException`

Pilha: exemplo resolvido

Operação	Pilha (topo à direita)	topo	Devolve
push(12) , push(7) , push(30)	12 7 30	2	
pop()	12 7	1	30
push(4)	12 7 4	2	
peek()	12 7 4	2	4 (não tira)
pop()	12 7	1	4
pop()	12	0	7
tamanho()	12	0	1

Fila circular: as duas linhas-chave

```
// enqueue
dados[fim] = v;
fim = (fim + 1) % dados.length;
tam++;

// dequeue
v = dados[inicio];
inicio = (inicio + 1) % dados.length;
tam--;
```

- O `%` faz o índice voltar para `0` depois da última posição
- Em outras versões, `inicio` = `frente` e `tam` = `tamanho`

Fila circular: exemplo resolvido (capacidade 4)

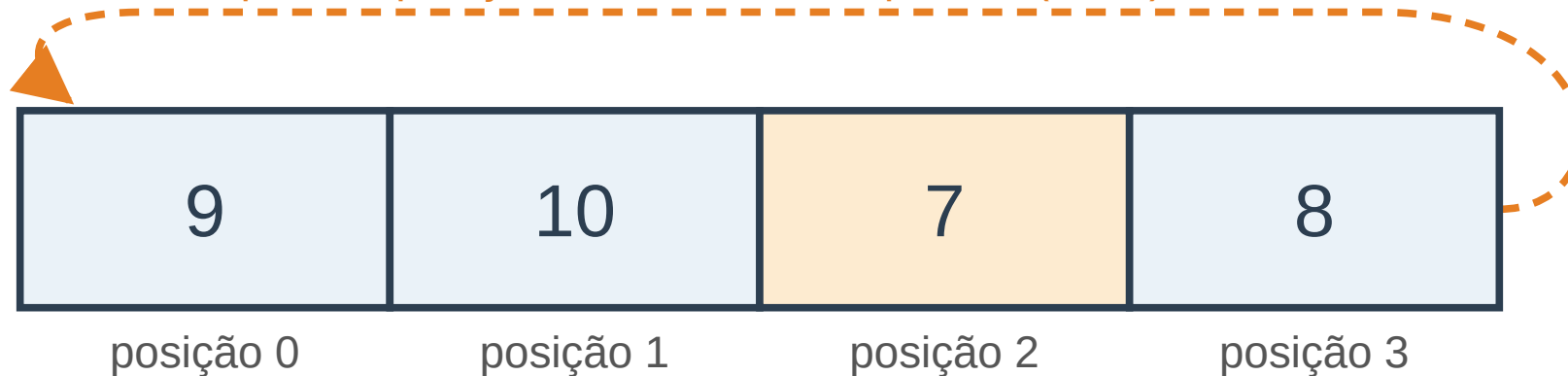
Operação	inicio	fim	tam	array	Devolve
enqueue 5, 6, 7	0	3	3	[5, 6, 7, _]	
dequeue()	1	3	2	[5, 6, 7, _]	5
enqueue(8)	1	0	3	[5, 6, 7, 8]	
enqueue(9)	1	1	4	[9, 6, 7, 8]	
dequeue()	2	1	3	[9, 6, 7, 8]	6
enqueue(10)	2	2	4	[9, 10, 7, 8]	

- $(3 + 1) \% 4 = 0$: o fim deu a volta
- dequeue **não apaga** o valor: só avança inicio
- Ordem de saída: **7, 8, 9, 10**

A fila cheia

Fila circular com capacidade 4, depois de enqueue(10)

depois da posição 3, o índice volta para 0: $(3 + 1) \% 4 = 0$



início = 2 e fim = 2

tam = 4: fila cheia. Ordem de saída: 7, 8, 9, 10 (sai sempre quem está em início).

Armadilhas da Etapa 7

- Com capacidade 4, **fim** **nunca vale 4**: depois do 3 vem o 0
- `inicio == fim` pode ser fila **cheia** ou **vazia** — por isso existe `tam`
- `dequeue` não apaga; o valor antigo fica no array, **fora da fila**
- `peek` **olha**; `pop` e `dequeue` **tiram**
- Pilha tira o **último** que entrou; fila tira o **primeiro**

Etapa 8

Bubble, Selection e Insertion

Os três algoritmos

	Uma passagem faz...	Melhor	Pior	Estável	Trocas
Bubble	troca vizinhos ; o maior vai para o fim	$O(n)$ com trocou	$O(n^2)$	sim	muitas
Selection	acha o menor e faz uma troca	$O(n^2)$	$O(n^2)$	não	$\leq n - 1$
Insertion	insere a chave no prefixo, deslocando os maiores	$O(n)$	$O(n^2)$	sim	deslocamentos

- Os três são **in-place** ($O(1)$ de memória extra)
- Selection: **$n(n - 1)/2$** comparações **sempre** ($n = 5 \rightarrow 10$; $n = 6 \rightarrow 15$)

Uma passagem de cada, em {6, 2, 9, 1, 5}

Bubble, i = 0 (compara vizinhos):

j	par	troca?	vetor
0	6 e 2	sim	[2, 6, 9, 1, 5]
1	6 e 9	não	[2, 6, 9, 1, 5]
2	9 e 1	sim	[2, 6, 1, 9, 5]
3	9 e 5	sim	[2, 6, 1, 5, 9]

Selection, i = 0: menor é o 1 (posição 3) → **uma** troca → [1, 2, 9, 6, 5]

Insertion, i = 1: chave = 2, desloca o 6 → [2, 6, 9, 1, 5]

Como escolher

- **Quase ordenado** ou dados chegando **um a um** → **Insertion**
- Memória que **gasta com gravação** (flash, EEPROM) → **Selection** (poucas trocas)
- Empates precisam **manter a ordem** → **estável**: Insertion ou Bubble, **nunca** Selection
- **Muitos dados** → nenhum dos três (todos $O(n^2)$)

Instabilidade do Selection: {5a, 5b, 2} → troca o 2 com o 5a → [2, 5b, 5a] : os iguais se inverteram

Armadilhas da Etapa 8

- Bubble: a passagem vai até $j < n - 1 - i$ — o fim já ordenado não é comparado
- Bubble leva o **maior** para o fim; o menor anda **uma posição** por passagem
- Selection **anota** o índice do menor e troca **uma vez**, só se `minIdx != i`
- Insertion **desloca**, não troca; a `chave` entra no buraco
- Bubble **sem** a flag `trocou` faz todas as passagens, mesmo ordenado
- "Todos são $O(n^2)$, então gastam o mesmo tempo" — **falso**: a notação descarta constantes

Etapa 9

Simulado

Simulado: como fazer

1. **Sem consulta** — feche texto, slides e caderno
2. **45 minutos**, sem interrupção
3. **Discursiva à mão, no papel, com caneta**
 - a prova é manuscrita: no papel ninguém avisa do `;`, da `}` ou do `return` esquecido
4. Código nas objetivas? **Tabela de variáveis** no papel
5. **Só depois**, corrija com a "**Resolução comentada do simulado**", no fim do texto de revisão
6. Para cada erro, **volte à etapa** e refaça as perguntas de fixação

Na véspera da prova

- Releia estes slides, **uma etapa por vez**
- Refaça **à mão** um método de lista, um de pilha e um de fila
- Refaça **um traçado** de cada: recursão, fila circular, uma passagem de ordenação
- Durma bem: traçado exige atenção

Dúvidas? Escreva nos comentários do material no Classroom, dizendo a **etapa** e a **pergunta**.

Boa prova!